# SIMATS Python Programming – Assignment 2

**Control Flow, Strings, Functions and Recursion – Predict, Design, Implement, Measure and Redesign**

**Student name:** Novia sri .A **Register number:** 1924260186 **Course:** CSA08 – CO2

Python 3.12.3 | Answers for all 7 questions | 50 marks

**GitHub Repository Link:** https://github.com/lunanoviasri-hue/CSA08-1924260186-Assignment2

All programs were executed and the outputs, operation counts and timings below come from those runs. Timings vary from one computer to another – re-run the programs on your own machine and replace the timing values with your own before submission.

## Q1 – Decisions with Many Boundaries (6 Marks)

**Problem:** ≤100 units → subsidy; 101–200 units → partial subsidy only with a ration card; above 200 units → no subsidy unless a member is above 80 years.

### 1. Predictions (before running any code)

| Units | Ration card | Member >80 | Predicted decision | Reason |
|---|---|---|---|---|
| 100 | No | No | FULL subsidy | boundary: 100 or less |
| 100 | Yes | No | FULL subsidy | boundary: 100 or less (ration card irrelevant) |
| 101 | No | No | NO subsidy | first unit of middle slab, no card |
| 101 | Yes | No | PARTIAL subsidy | first unit of middle slab, has card |
| 200 | Yes | No | PARTIAL subsidy | last unit of middle slab |
| 201 | No | No | NO subsidy | above 200, nobody above 80 |
| 201 | No | Yes | FULL subsidy (senior) | above 200 but member above 80 |
| 150 | Yes | No | PARTIAL subsidy | inside the middle slab |

### 2. Why separate `if` can give more than one result, but `if-elif-else` gives exactly one

Every separate `if` is an independent test: Python evaluates all of them, so each one whose condition is true runs. In the faulty version the second rule was written as `units <= 200` and `ration_card` (lower bound 101 forgotten), so a household with 100 units and a ration card satisfies both the first rule and the second rule and receives **two** subsidies. In `if-elif-else` the conditions form one chain: the first true branch runs and all remaining branches are skipped, so exactly one decision is produced, and the `else` branch covers everything not handled earlier.

### 3. Design decision – nested if or combined Boolean conditions?

I chose a **flat if-elif-else chain with chained comparisons and one combined Boolean/conditional expression per slab** (`101 <= units <= 200`). Nested `if` statements would add indentation levels and scatter the slab limits over several places. With the flat chain each slab is one readable line, a new slab is one more `elif`, and the order of conditions makes the boundaries (100/101, 200/201) visible and easy to test.

### 4. Alternative concept and complexity analysis

- **match-case (3.10+):** `case u if u <= 100` … – same decisions (verified for every units value 0–399 and all card/age combinations). It checks cases top to bottom, so time per decision is O(k) for k slabs, space O(1).

- **List of (upper_limit, decision) pairs searched in a loop:** time O(k) per decision (O(log k) if the sorted limits are searched with `bisect`), space O(k) for the table. Changing a slab means editing data, not code.
- **if-elif-else:** time O(k) worst case (O(1) for the first slab), space O(1).
- **Measured:** if-elif-else 0.145 µs, match-case 0.110 µs, table loop 0.139 µs per decision – practically identical for k = 3; the table only pays off when k is large or slabs change often.

# Q2 – Loops that Count Attempts (4 Marks)

**Problem:** three OTP attempts, then the account is locked for 30 minutes; a blank entry must not count as an attempt.

## 1. Designs

- **while loop:** `while attempts < 3`; the counter is increased only after the blank check, so blank input costs nothing.
- **for loop with range(3):** the loop variable is the attempt counter, so a blank entry has to be consumed by an inner skip, otherwise it would burn an attempt.
- **break** – leaves the loop as soon as the OTP is correct. **continue** – skips a blank entry without counting it. **pass** – placeholder branch where nothing has to happen.
- **loop else:** the `else` of the loop runs only when the loop ended without `break`, i.e. after 3 wrong attempts → "account LOCKED for 30 minutes".

## 2. Answer the following

- **When is while more natural than for?** When the number of iterations is not known in advance and depends on a condition – here blank entries are free and the limit can change (security question). `for … range(3)` is more natural when exactly three tries are always made.
- **Bug if the counter is updated in the wrong place:** if `attempts += 1` is placed before the blank check, blank entries are counted – the demonstration at the end of the output shows 3 blank entries counted as 3 attempts, which locks the customer out unfairly. If it is placed after the success check but before `continue`/`break` is skipped, the counter can also fail to increase and cause an endless loop.

## 3. Changed condition – two extra attempts after a correct security answer

The limit is now a **run-time value** (3, or 5 after the security question). `range(3)` fixes the count when the loop starts, so I re-evaluated the design and chose the **while loop** with a variable `allowed` that is raised from 3 to 5 once. Output shows: security answer correct → attempts 4 and 5 are allowed (access granted at attempt 5); security answer wrong → locked after 3.

## 4. Alternative concept – recursion

`verify_recursive(entries, attempt, i)` calls itself with `attempt + 1` after a wrong OTP and with the same attempt after a blank. **Time:** O(a) for a attempts in all versions. **Stack space:** O(a) for recursion (one frame per attempt) versus O(1) for a loop. For a ≤ 5 the difference is negligible, but a loop is simpler and cannot hit the recursion limit.

# Q3 – Engineer a Vehicle Registration Number Validator (6 Marks)

**Rule:** 2 letters (state), 2 digits (district), 1 or 2 letters (series), 4 digits (number); normalise → validate → format as `TN 09 AB 1234`, using only string operations (no regular expressions).

### 1. Prediction for six inputs of my choice (two or more invalid)

| Input | Normalised (predicted) | Prediction | Observed |
|---|---|---|---|
| Tn09Ab1234 | TN09AB1234 | valid | valid |
| mh-12-de-1433 | MH12DE1433 | valid | valid |
| DL 8 CAF 5031 | DL8CAF5031 | INVALID (district has 1 digit, 3 series letters) | INVALID |
| TN09ABC1234 | TN09ABC1234 | INVALID (3 series letters, length 11) | INVALID |
| TN09AB123 | TN09AB123 | INVALID (only 3 digits at the end) | INVALID |
| T109AB1234 | T109AB1234 | INVALID (digit inside the state code) | INVALID |

### 2. Inquiry

- **Why are strings immutable in Python?** A string object can never be changed after creation (`text[0] = 'T'` raises `TypeError`). This makes strings safe to share, lets them be used as dictionary keys (their hash never changes), allows interning/caching, and avoids bugs where one part of a program changes a string that another part still uses.
- **Why does text.upper() not modify the original text?** Because of immutability, `upper()` builds and returns a **new** string; the original object stays unchanged (output: original still `'tn 09 ab 1234'`, new string is a different object). The result must be assigned: `text = text.upper()`.
- **s[-4:] and s[4:-4]:** for the 9-character number `'TN09A1234'`: `s[-4:] = '1234'` (last four characters), `s[4:-4] = 'A'` (series with 1 letter). For the 10-character number `'TN09AB1234'`: `s[-4:] = '1234'`, `s[4:-4] = 'AB'` (series with 2 letters). Negative slicing from the end therefore works for both lengths without branching.

### 3. Implementation summary

- `normalise(text)` (TODO 1): `text.replace(' ', '').replace('-', '').upper()`
- `is_valid(reg)` (TODO 2): length must be 9 or 10, then `reg[:2].isalpha()`, `reg[2:4].isdigit()`, `reg[4:-4].isalpha()` with length 1 or 2, and `reg[-4:].isdigit()`.
- `format_number(reg)`: `' '.join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])` → `TN 09 AB 1234`.
- The four given inputs produce exactly the expected output (valid, valid, INVALID, INVALID).

### 4. Alternative concept – regular expression

`re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg)` replaces the whole validation with one pattern. **Time:** both approaches are O(L) for a number of length L (slicing/`isalpha`/`isdigit` scan each character at most once; the regex engine matches the fixed pattern in one pass). Since L is at most 10 in practice, both are effectively constant. **Measured** on 10,000 numbers (same results for both): slicing 0.273 µs, regex 0.500 µs per check. **Readability / modification:** the regex is shorter and easier to change (edit one pattern) but harder to read for beginners and gives no hint which part failed; the slicing version is longer but self-explanatory, easy to debug, and each part can give its own error message.

## Q4 – Building Long Text: + or join()? (6 Marks)

**Problem:** a monthly SMS bill statement is built from more than 100,000 lines; build it with repeated +=, with `"".join()` and with an f-string per line, and measure.

## 1. Prediction

I predicted that `join()` would be faster, because a string is immutable: every `text += line` would have to create a brand-new string and copy everything built so far, giving up to O(n²) characters copied for n lines, whereas `join()` first calculates the total size and copies each part once, O(n).

## 2. Implementations (TODO 1 completed)

- `build_plus`: `text += p + ";"` inside a loop.
- `build_join` (TODO 1): `"".join([p + ";" for p in parts])` – one `str.join` call.
- `build_fstring`: `text += f"{p};"` for each line; extra method `build_stringio` (io.StringIO) for the alternative analysis.

The program prints `same text: True | length: 3048890`, exactly as in the expected output.

## 3. Measured execution time (three runs and average, time.perf_counter(), on my computer)

| Lines | Method | Run 1 (s) | Run 2 (s) | Run 3 (s) | Average (s) |
|---|---|---|---|---|---|
| 1,000 | += | 0.00007 | 0.00006 | 0.00006 | 0.00006 |
| 1,000 | join() | 0.00006 | 0.00003 | 0.00003 | 0.00004 |
| 1,000 | f-string += | 0.00006 | 0.00006 | 0.00006 | 0.00006 |
| 1,000 | StringIO | 0.00005 | 0.00004 | 0.00004 | 0.00005 |
| 10,000 | += | 0.00063 | 0.00064 | 0.00055 | 0.00061 |
| 10,000 | join() | 0.00059 | 0.00042 | 0.00036 | 0.00046 |
| 10,000 | f-string += | 0.00060 | 0.00055 | 0.00056 | 0.00057 |
| 10,000 | StringIO | 0.00060 | 0.00052 | 0.00053 | 0.00055 |
| 200,000 | += | 0.01328 | 0.01281 | 0.01087 | 0.01232 |
| 200,000 | join() | 0.01763 | 0.01536 | 0.01447 | 0.01582 |
| 200,000 | f-string += | 0.01149 | 0.01178 | 0.01203 | 0.01177 |
| 200,000 | StringIO | 0.01043 | 0.01109 | 0.01045 | 0.01066 |

## 4. Inquiry – explaining the results using immutability

**Honest observation:** my prediction was only partly confirmed. At 1,000 and 10,000 lines `join()` was faster, but at 200,000 lines += (0.0123 s) was even slightly faster than `join()` (0.0158 s) on my machine, and no quadratic blow-up appeared. The reason is that **CPython sometimes optimises +=**: when the left-hand string has only one reference, the interpreter can resize it in place instead of copying, so repeated += behaves almost like appending. This is an implementation detail, not a language guarantee – it does not work if another reference to the string exists (for example when the string is stored in an object attribute or list), and it is absent in other Python implementations such as PyPy or Jython. Without the optimisation, immutability means each += allocates a new string and copies all earlier characters (up to O(n²) characters copied in total). `join()` is guaranteed O(n) in every implementation and every situation, so it **remains the recommended practice**. The first join() run at 200,000 lines (0.0176 s) was slower than the next two (0.0154 s, 0.0145 s), a warm-up effect; the average over three runs is what counts.

### 5. Alternative concept and complexity analysis – write line by line to a file or io.StringIO

- **Time:** join() is O(n) in total characters; StringIO/file writing is also O(n) (amortised); repeated += is up to $O(n^2)$ characters copied in the worst case (O(n) with the CPython optimisation).
- **Memory:** join() needs the list of parts and the final string (about 2n); += holds the growing string plus a temporary copy while resizing; StringIO holds a buffer of about n; writing directly to a file needs only a small buffer, O(1) extra memory – the best choice for very large statements.
- **Measured at 200,000 lines:** StringIO 0.0107 s (fastest), f-string += 0.0118 s, += 0.0123 s, join() 0.0158 s.

## Q5 – Functions, Parameters and Scope (7 Marks)

**Problem:** `add_item(item, order=[])` leaks items of the previous customer into the next customer's order.

### 1. Explaining the bug (using the given principle)

A default value is created once, when the function is defined – not each time the function is called. The empty list `[]` in `order=[]` is created a single time and stored in `add_item_buggy.__defaults__`. Every call that does not pass `order` appends to that **same** list object. The output proves it: after the two calls, `__defaults__` is `(['Tea', 'Vada'],)` – the "default" list has grown, and both calls returned the same object, so both printed `['Tea', 'Vada']`.

**Correct design (TODO 1):** use `None` as the default (immutable), and create the list inside the function: `if order is None: order = []`. Now each call without an order gets a new list; output `['Tea'] ['Vada']`, and `add_item.__defaults__` stays `(None,)`.

### 2. Design questions

- **Positional, keyword and default parameters:** use positional for the few obvious required arguments in a natural order (`price('Tea', 2)`); use keyword arguments when there are several values or the meaning is unclear, or to skip options (`price(qty=3, item='Vada', gst_rate=0.18)`); use default parameters for optional values with a sensible standard (`gst_rate=0.05`) – but only **immutable** defaults (numbers, strings, None).
- **Returning more than one value:** return a tuple and unpack it: `return total, gst, grand` then `total, gst, grand = bill([20, 15, 30])` → total=65, GST=3.25, grand total=68.25.
- **local, global and nonlocal:** a local name exists only inside the function (`local_demo()` returns 99, the global `counter` stays 0); a global name lives at module level; nonlocal refers to a variable of the enclosing function in nested functions (the token machine closure counts 1, 2, 3).
- **Why global is needed and why it is avoided:** an assignment inside a function creates a local name by default, so `counter += 1` without `global` raises `UnboundLocalError` (shown in the output); `global counter` tells Python to rebind the module-level name. It is usually avoided because any function can silently change shared state, which makes bugs hard to find, functions hard to test and reuse, and code unsafe in multi-threaded programs; passing parameters and returning values is clearer.

### 3. Test – three customers in sequence

Customer 1 → `['Tea', 'Vada']`, Customer 2 → `['Coffee']`, Customer 3 → `['Idli', 'Dosa', 'Juice']`: each order is independent. With the buggy function the same three customers produce `['Tea','Vada','Tea','Vada']`, then `[..., 'Coffee']`, then `[..., 'Idli']` – the orders pile up.

### 4. Alternative concept and complexity analysis

- **Caller always passes the list explicitly:** no hidden state, but every caller must remember to create a new list – a reminder burden on the caller.
- **Closure (`make_order_taker()`):** every customer gets an inner function with its own private list; output `['Tea'] ['Tea', 'Vada'] ['Coffee']` for two customers – independent.
- **Time:** O(1) amortised per item in all designs (`list.append`; the closure version returns a copy, so its output step is O(m) for m items).
- **Space per customer:** O(m) for m items in each design; the default-argument bug shares one list so its space is O(total items of all customers) and grows forever; the closure adds one small function object and cell per customer.

## Q6 – Recursion and Its Limits (8 Marks)

**Problem:** number of ways to climb n temple steps taking 1 or 2 steps at a time (staircases up to 1,000 steps).

### 1. Recursive design

- **Base case:** `n <= 1 → return 1` (one way to climb 0 or 1 step).
- **Recursive case:** `ways(n) = ways(n-1) + ways(n-2)` – the last move was either one step (from n-1) or two steps (from n-2).

### 2. Call tree for n = 5 and why plain recursion repeats work

| Sub-problem | ways(5) is computed through… | Times evaluated for n = 5 |
|---|---|---|
| ways(5) | root | 1 |
| ways(4) | ways(5) | 1 |
| ways(3) | ways(5) and ways(4) | 2 |
| ways(2) | ways(4) and twice via ways(3) | 3 |
| ways(1) | leaf | 5 |
| ways(0) | leaf | 3 |

The program prints the whole tree (15 calls for n = 5, see the output). `ways(3)` is computed twice and `ways(2)` three times, because each call starts again from scratch. For n = 5 there are only 6 different sub-problems but 15 calls; this waste doubles roughly with every extra step.

### 3. Memoisation

A dictionary or `@lru_cache` stores each answer the first time it is computed; later calls with the same n return the stored value immediately. Each of the n + 1 sub-problems is computed once, so the call tree collapses to a single chain: **26 calls for n = 25** instead of **242,785**.

### 4. Correctness check (manual answers, n = 1 to 6)

| n | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| ways(n) by hand | 1 | 2 | 3 | 5 | 8 | 13 |
| program (naive, memo, loop) | 1 | 2 | 3 | 5 | 8 | 13 |

By hand: ways(2) = ways(1) + ways(0) = 2; ways(3) = 2 + 1 = 3; ways(4) = 3 + 2 = 5; ways(5) = 5 + 3 = 8; ways(6) = 8 + 5 = 13 – all match the program.

### 5. Growth investigation – function calls

| n | Plain recursion (calls) | Memoised (calls) | Predicted 2·ways(n) − 1 |
|---|---|---|---|
| 10 | 177 | 11 | 177 |
| 20 | 21,891 | 21 | 21,891 |
| 25 | 242,785 | 26 | 242,785 |
| 30 | 2,692,537 | 31 | 2,692,537 |

**Prediction vs result:** I predicted that the call count of the plain version multiplies by about $\varphi \approx 1.618$ for every extra step, i.e. $O(\varphi^n)$. The measured counts confirm it exactly: calls(n) = 2·ways(n) − 1 (21,891 → 242,785 for five more steps is a factor 11.1 ≈ $1.618^5$). The memoised version needs n + 1 calls: linear growth.

### 6. Deployment decision – n = 1,000

The memoised version for n = 1,000 stops with `RecursionError: maximum recursion depth` exceeded, because `sys.getrecursionlimit()` is 1000 and the recursion needs about 1000 nested calls. Raising the limit to 5000 works (209-digit answer) but only moves the limit: a deeper recursion can crash the Python process with a real C-stack overflow, and memory for 1000+ frames is wasted. **My decision: rewrite the function using a loop** for deployment (constant stack, O(1) extra space, 0.000035 s for n = 1000), and keep the memoised recursive version only as a readable reference with small n (and as a cross-check: both return the same value for n = 1000). I do not rely on raising the recursion limit.

### 7. Alternative concept and complexity analysis – loop keeping the last two values

| Approach | Time | Space | n = 1000 (average of 3 runs) |
|---|---|---|---|
| Plain recursion | $O(\varphi^n)$ | O(n) stack | impossible (~$10^{209}$ calls) |
| Memoised recursion | O(n) | O(n) cache + O(n) stack | 0.000255 s (needs raised limit) |
| Loop, last two values | O(n) | O(1) auxiliary | 0.000035 s |

## Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 Marks)

**Selected program:** Q6 – counting ways to climb n temple steps with plain recursion (`ways_naive`). Its limitation: it cannot handle larger staircases – at n = 32 it already needs 7 million calls and for n = 1,000 it would never finish.

## A. Diagnose the bottleneck (2 marks)

- **Where most work is done:** in the repeated re-evaluation of the same small sub-problems near the bottom of the call tree. For n = 20 there are only **21 distinct sub-problems** but **21,891 calls**; `ways(1)` is evaluated 6,765 times and `ways(2)` 4,181 times.
- **Which count grows with the input:** the number of function calls: 177 (n=10), 21,891 (n=20), 242,785 (n=25), 2,692,537 (n=30) – multiplied by ≈1.618 per step, $O(\varphi^n)$.
- **Unnecessary repetition:** yes – 21,870 of the 21,891 calls for n = 20 repeat work that was already done (all calls minus the 21 distinct sub-problems).

## B. Design objective (2 marks)

**Measurable objective:** compute ways(n) with a number of operations that grows **linearly** (at most n − 1 additions, instead of 2,692,537 calls at n = 30), run n = 30 in under 0.1 ms (original: 150 ms), and run n = 1,000 and n = 10,000 without RecursionError and without extra memory growth.

## C. Design of the revised solution (4 marks)

- **What changed:** the top-down recursion was replaced by a bottom-up loop that starts from the base values and builds up to n.
- **Property exploited:** overlapping sub-problems and optimal substructure – ways(n) depends only on the two previous values, so older values never need to be kept or recomputed.
- **Extra memory:** none – two variables, O(1) auxiliary space (the memoised version would need O(n) cache and O(n) stack).
- **New trade-offs:** the code no longer mirrors the mathematical definition as directly; intermediate results are not available for later queries unless stored (memoisation is better if many different n are asked repeatedly); for huge n (10,000) the numbers have thousands of digits, so each addition is no longer O(1).

```
ways_loop(n):
    a <- 1                    # ways(0)
    b <- 1                    # ways(1)
    repeat n - 1 times:
        (a, b) <- (b, a + b)  # slide the window of two values
    return b
```

## D. Predict the improvement (2 marks)

**Testable prediction:** the operation count will fall from 2·ways(n) − 1 function calls to exactly n − 1 additions (n = 30: 2,692,537 → 29, about 93,000 times fewer), and the execution time will fall by at least a factor of 10,000 for n = 30. The improvement is expected in the **operation count** (primary measure) and the **execution time** (secondary measure).

## E. Implement and validate the redesign (2 marks)

Same inputs, operation counts and execution-time measurements for the original and the redesign:

| n | Original calls | Original time (s) | Redesign additions | Redesign time (s) | Same answer | Speed-up |
|---|---|---|---|---|---|---|
| 10 | 177 | 0.000012 | 9 | 0.0000032 | True | 4× |
| 20 | 21,891 | 0.001198 | 19 | 0.0000028 | True | 432× |
| 25 | 242,785 | 0.013457 | 24 | 0.0000032 | True | 4,190× |
| 30 | 2,692,537 | 0.150371 | 29 | 0.0000099 | True | 15,257× |
| 32 | 7,049,155 | 0.388292 | 31 | 0.0000082 | True | 47,157× |

The redesign alone also handles n = 100 (21 digits, 0.000009 s), n = 1,000 (209 digits, 0.000062 s) and n = 10,000 (2,090 digits, 0.001357 s), where the original cannot finish. The graph below plots the operation counts (log scale) from the same program.




Q7 – growth of work: original recursion (red) vs redesigned loop (green)

## F. Defend the final design (2 marks)

- **Objective achieved?** Yes – the operation count is exactly n − 1 additions (29 for n = 30), n = 30 runs in 0.0000099 s, and n = 1,000 and n = 10,000 run without any RecursionError.
- **Size of the improvement:** 93,000× fewer operations and 15,257× faster at n = 30; the gap keeps growing exponentially with n.
- **New costs / trade-offs:** less direct match to the recurrence, no reusable cache of sub-results; very large n makes the big-integer additions themselves cost more.
- **When the original may still be preferable:** for very small n (n ≤ 10: 177 calls, about 0.00001 s – no practical difference), when clarity and teaching value matter more than speed, or when the problem has several branches that cannot be reduced to a two-value window. For repeated queries with many different n, the memoised version is a better compromise than either.

**Conclusion:** the diagnosis (repeated sub-problems, exponential call growth) was confirmed by the counts, the redesign met every measurable objective, and the evidence for both versions is supplied with the raw numbers above.

## Overall Big-O summary

| Question | Approach | Time | Extra space |
|---|---|---|---|
| Q1 | if-elif-else / match-case per decision | O(k) for k slabs | O(1) |
| Q1 | table of (limit, decision) pairs | O(k) (O(log k) with bisect) | O(k) |
| Q2 | attempt loop (while / for) | O(a) | O(1) |
| Q2 | recursive attempts | O(a) | O(a) stack |
| Q3 | slicing validation or regex | O(L) | O(L) |
| Q4 | += (worst case) / join / StringIO | $O(n^2)$ copied / O(n) / O(n) | O(1)–O(n) / O(n) / O(n) |
| Q5 | add_item with None default | O(1) per item | O(m) per customer |
| Q6 | plain recursion / memoised / loop | $O(\varphi^n)$ / O(n) / O(n) | O(n) / O(n) / O(1) |

## Submission checklist

- Create repository named `CSA08-1924260186-Assignment2`.
- Create Task01 … Task07 folders and place code, output files and screenshots in the matching folders.
- Add README.md with name, register number, Python version, libraries, execution commands and task summaries.
- Make progressive Git commits, with at least one commit per task (not all files in one commit).
- Include report explanations, source code, input/output evidence, raw operation counts and timings, tables and graphs.
- Submit the accessible GitHub link and the PDF report through the assignment portal.

## Viva quick revision

- **Why can separate if statements give two results?** Each if is tested independently; elif/else skips the rest after the first true branch.
- **What does loop else do?** It runs only if the loop finished without break – used to lock the account after 3 wrong OTPs.
- **Why are strings immutable?** Safe sharing, hashing and caching; methods like upper() return new strings.
- **Why join() and not += ?** join() is guaranteed O(n); += is only fast because of a CPython optimisation that is not guaranteed.
- **Why order=None?** A default value is created once at function definition, so a mutable default is shared by all calls.
- **Why is plain recursion slow?** It repeats sub-problems: calls = 2·ways(n) − 1, $O(\varphi^n)$; memoisation or a loop makes it O(n).
- **Why RecursionError at n = 1000?** The recursion limit is 1000 frames; a loop uses O(1) stack.
- **What did Q7 change?** Recursion replaced by a two-variable loop: 2,692,537 calls → 29 additions at n = 30.

# SIMATS Python Assignment 2 | Output Screenshots (VS Code)

Student: Novia sri .A  Register number: 1924260186  – Q1 to Q7: code and terminal output

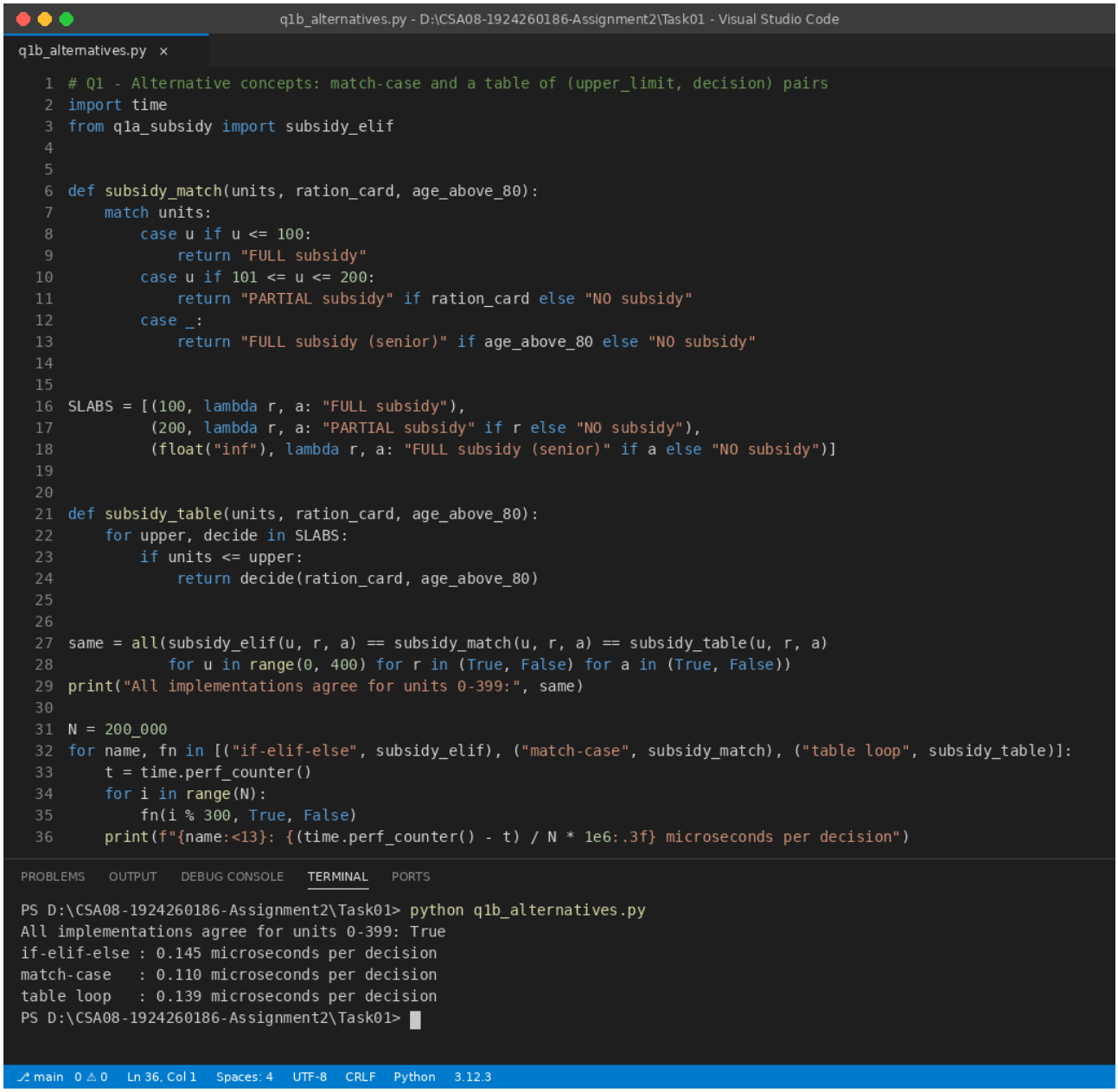
### Q1 - Decisions with many boundaries (subsidy eligibility) – q1a_subsidy.py – separate if vs if-elif-else

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



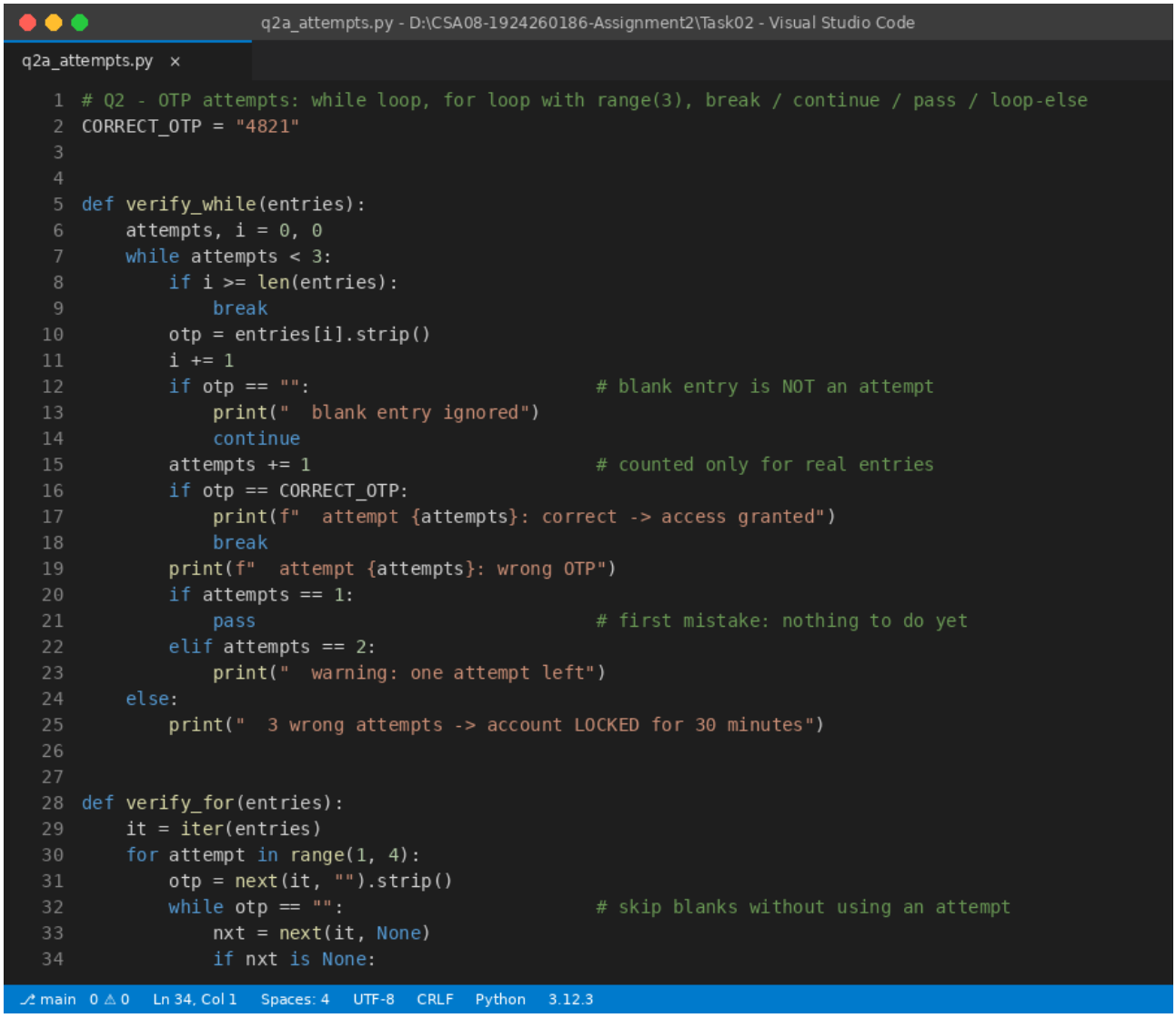
## Q1 - Decisions with many boundaries (subsidy eligibility) – q1b_alternatives.py – match-case, table loop, timing

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q2 - Loops that count attempts (OTP verification) – q2a_attempts.py – while and for loops, break/continue/pass/else (part 1/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q2 - Loops that count attempts (OTP verification) – q2a_attempts.py – while and for loops, break/continue/pass/else (part 2/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q2a_attempts.py - D:\CSA08-1924260186-Assignment2\Task02 - Visual Studio Code

q2a_attempts.py

```
                return
            print("  blank entry ignored")
            otp = nxt.strip()
        if otp == CORRECT_OTP:
            print(f"  attempt {attempt}: correct -> access granted")
            break
        print(f"  attempt {attempt}: wrong OTP")
    else:
        print("  3 wrong attempts -> account LOCKED for 30 minutes")


tests = {"A: blank, wrong, blank, correct": ["", "1111", "", "4821"],
         "B: three wrong OTPs": ["1111", "2222", "3333"]}
for title, data in tests.items():
    print(f"=== {title} ===")
    print("while loop:")
    verify_while(data)
    print("for loop with range(3):")
    verify_for(data)
```

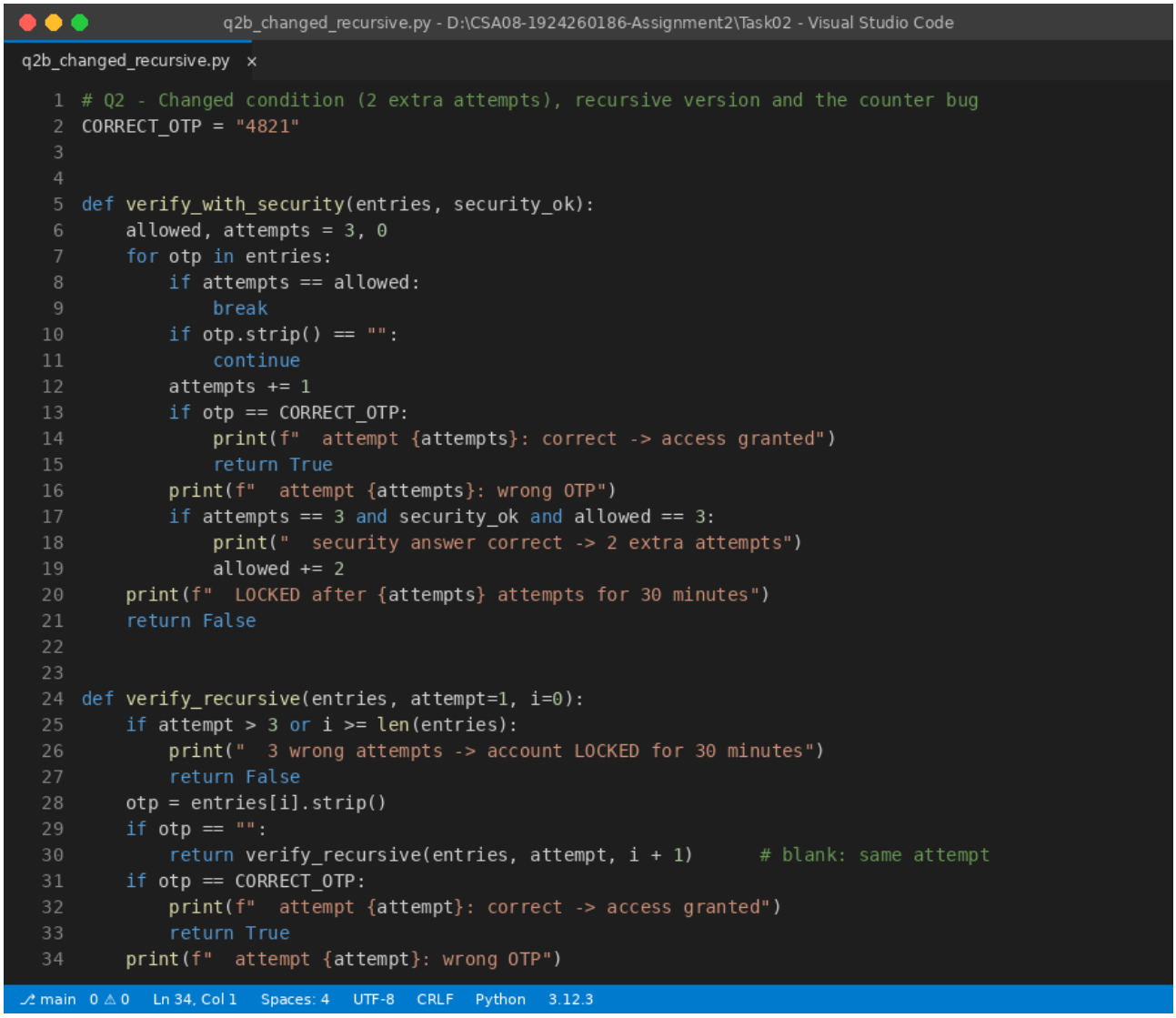
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task02> python q2a_attempts.py
=== A: blank, wrong, blank, correct ===
while loop:
  blank entry ignored
  attempt 1: wrong OTP
  blank entry ignored
  attempt 2: correct -> access granted
for loop with range(3):
  blank entry ignored
  attempt 1: wrong OTP
  blank entry ignored
  attempt 2: correct -> access granted
=== B: three wrong OTPs ===
while loop:
  attempt 1: wrong OTP
  attempt 2: wrong OTP
  warning: one attempt left
  attempt 3: wrong OTP
  3 wrong attempts -> account LOCKED for 30 minutes
for loop with range(3):
  attempt 1: wrong OTP
  attempt 2: wrong OTP
  attempt 3: wrong OTP
  3 wrong attempts -> account LOCKED for 30 minutes
PS D:\CSA08-1924260186-Assignment2\Task02>
```

main 0 ⚠ 0 Ln 53, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

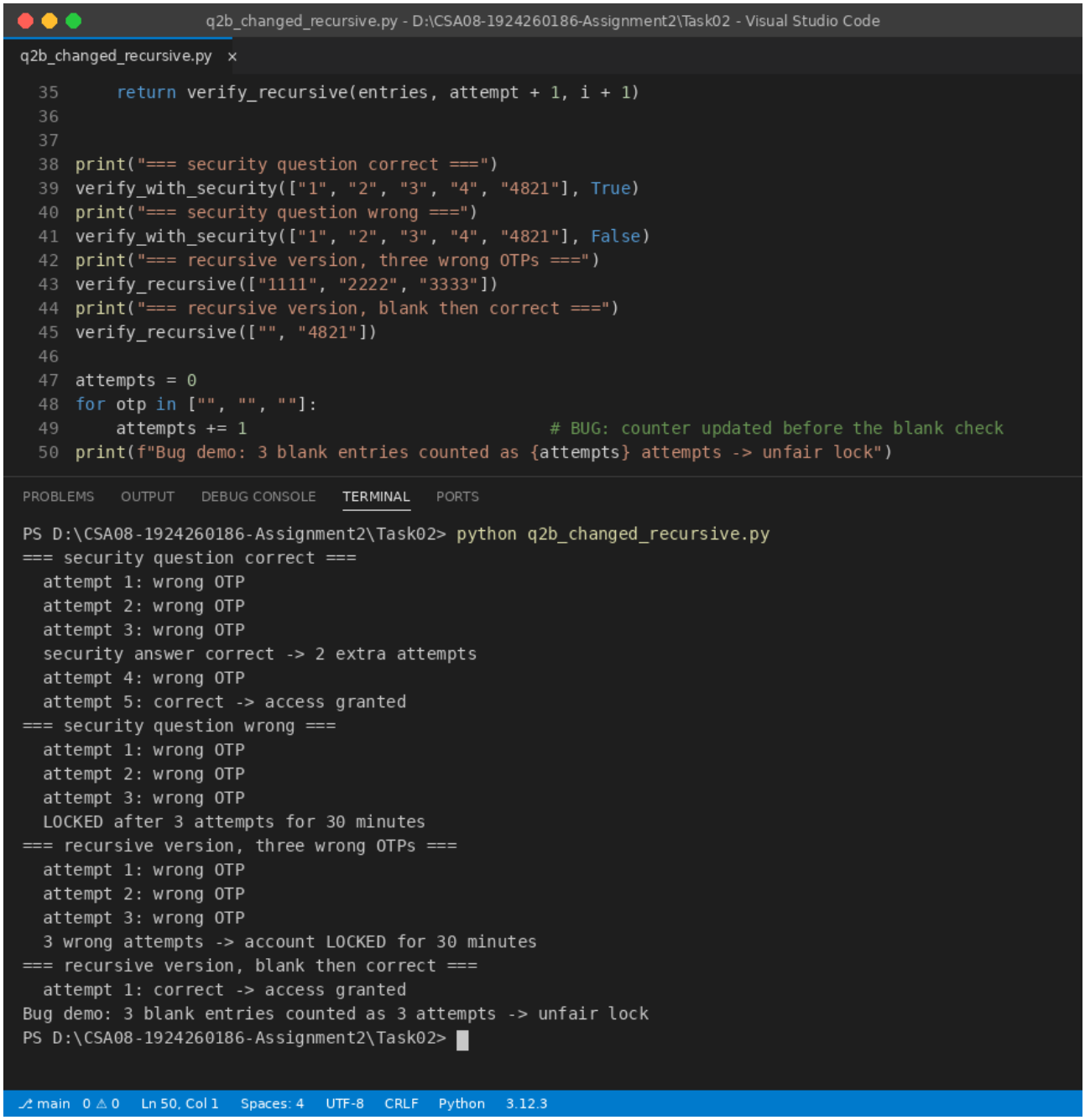
## Q2 - Loops that count attempts (OTP verification) – q2b_changed_recursive.py – changed condition, recursion, counter bug (part 1/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q2 - Loops that count attempts (OTP verification) – q2b_changed_recursive.py – changed condition, recursion, counter bug (part 2/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q3 - Vehicle registration number validator – q3a_vehicle.py – normalise, validate, format

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q3a_vehicle.py - D:\CSA08-1924260186-Assignment2\Task03 - Visual Studio Code

q3a_vehicle.py

```python
# Q3 - Vehicle registration validator (string operations only, no regular expressions)

def normalise(text):
    """'tn 09 ab 1234' -> 'TN09AB1234'"""
    return text.replace(" ", "").replace("-", "").upper()          # TODO 1


def is_valid(reg):
    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits."""
    if len(reg) not in (9, 10):                                    # TODO 2
        return False
    state, district = reg[:2], reg[2:4]
    series, number = reg[4:-4], reg[-4:]
    return (state.isalpha() and district.isdigit()
            and series.isalpha() and len(series) in (1, 2)
            and number.isdigit())


def format_number(reg):
    """'TN09AB1234' -> 'TN 09 AB 1234'"""
    return " ".join([reg[:2], reg[2:4], reg[4:-4], reg[-4:]])


if __name__ == "__main__":
    for raw in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]:
        reg = normalise(raw)
        print(f"{raw:15} -> {reg:11} {'valid' if is_valid(reg) else 'INVALID'}")
    print()
    for raw in ["tn 09 ab 1234", "KA-01-M-0007"]:
        print(f"{raw:15} -> {format_number(normalise(raw))}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task03> python q3a_vehicle.py
tn 09 ab 1234   -> TN09AB1234  valid
KA-01-M-0007    -> KA01M0007   valid
TN9AB1234       -> TN9AB1234   INVALID
TN 22 1234 AB   -> TN221234AB  INVALID

tn 09 ab 1234   -> TN 09 AB 1234
KA-01-M-0007    -> KA 01 M 0007
PS D:\CSA08-1924260186-Assignment2\Task03>
```

main 0 ⚠ 0 Ln 30, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

## Q3 - Vehicle registration number validator – q3b_inquiry.py – predictions, slicing, immutability, regex

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q3b_inquiry.py - D:\CSA08-1924260186-Assignment2\Task03 - Visual Studio Code

q3b_inquiry.py

```python
# Q3 - Prediction, slicing, immutability and regex comparison
import re, time
from q3a_vehicle import normalise, is_valid

mine = [("Tn09Ab1234", True), ("mh-12-de-1433", True), ("DL 8 CAF 5031", False),
        ("TN09ABC1234", False), ("TN09AB123", False), ("T109AB1234", False)]
for raw, pred in mine:
    reg = normalise(raw)
    got = is_valid(reg)
    print(f"{raw:14} -> {reg:12} predicted={'valid' if pred else 'INVALID':7} "
          f"observed={'valid' if got else 'INVALID':7} {'OK' if pred == got else 'MISMATCH'}")

for s in ["TN09A1234", "TN09AB1234"]:
    print(f"s={s!r} len={len(s)}: s[-4:]={s[-4:]!r}  s[4:-4]={s[4:-4]!r}")

text = "tn 09 ab 1234"
up = text.upper()
print("original:", repr(text), "| new string:", repr(up), "| same object?", text is up)
try:
    text[0] = "T"
except TypeError as e:
    print("text[0] = 'T' ->", type(e).__name__ + ":", e)


def is_valid_regex(reg):
    return re.fullmatch(r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}", reg) is not None


samples = [normalise(x) for x in ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"] * 2500]
print("same results:", all(is_valid(s) == is_valid_regex(s) for s in samples))
for name, fn in [("slicing", is_valid), ("regex", is_valid_regex)]:
    t = time.perf_counter()
    for s in samples:
        fn(s)
    print(f"{name:8}: {(time.perf_counter() - t) / len(samples) * 1e6:.3f} microseconds per check")
```

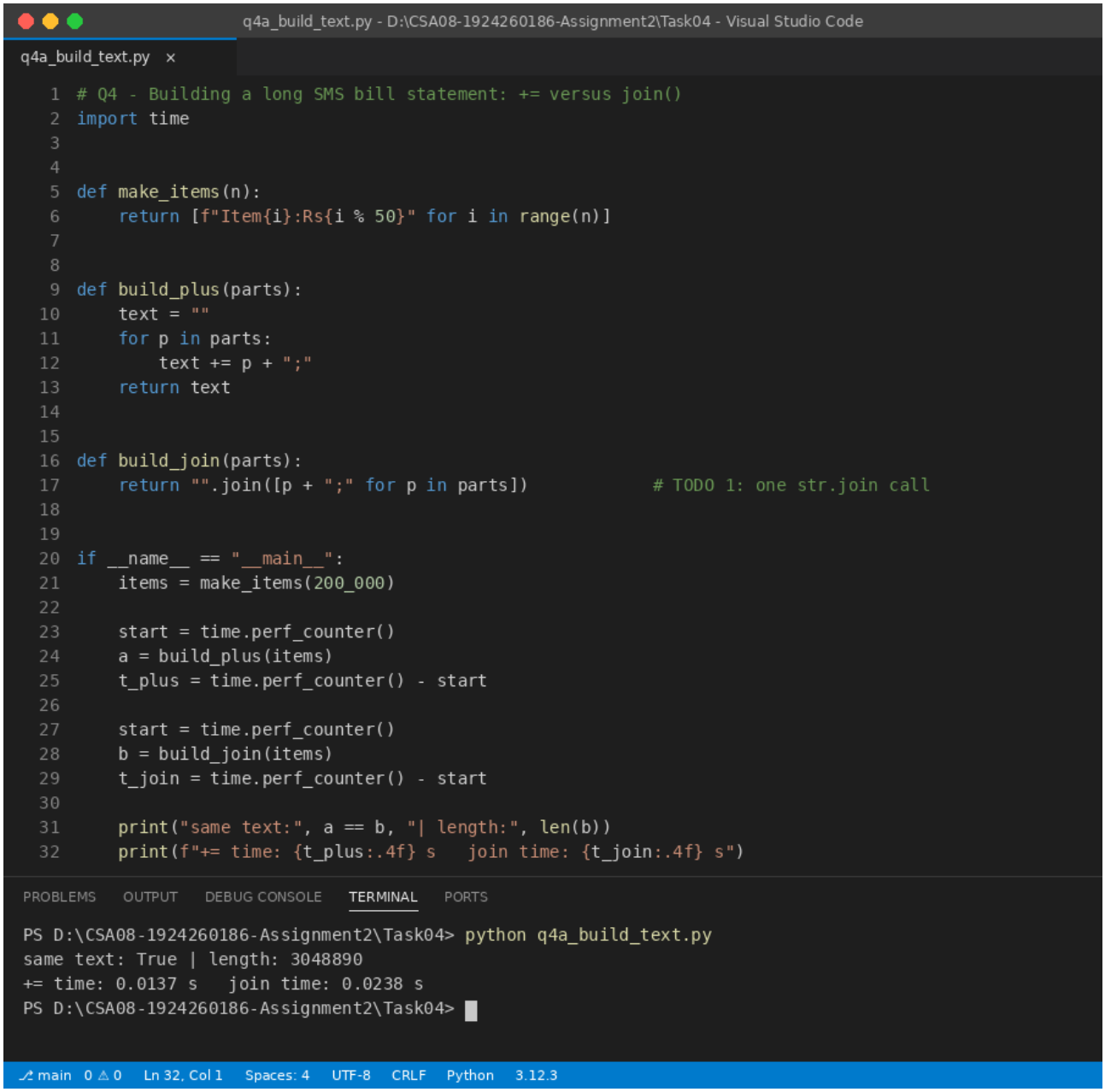
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task03> python q3b_inquiry.py
Tn09Ab1234     -> TN09AB1234   predicted=valid   observed=valid   OK
mh-12-de-1433  -> MH12DE1433   predicted=valid   observed=valid   OK
DL 8 CAF 5031  -> DL8CAF5031   predicted=INVALID observed=INVALID OK
TN09ABC1234    -> TN09ABC1234  predicted=INVALID observed=INVALID OK
TN09AB123      -> TN09AB123    predicted=INVALID observed=INVALID OK
T109AB1234     -> T109AB1234   predicted=INVALID observed=INVALID OK
s='TN09A1234' len=9: s[-4:]='1234'  s[4:-4]='A'
s='TN09AB1234' len=10: s[-4:]='1234'  s[4:-4]='AB'
original: 'tn 09 ab 1234' | new string: 'TN 09 AB 1234' | same object? False
text[0] = 'T' -> TypeError: 'str' object does not support item assignment
same results: True
slicing : 0.273 microseconds per check
regex   : 0.500 microseconds per check
PS D:\CSA08-1924260186-Assignment2\Task03>
```

main 0 ⚠ 0 Ln 35, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

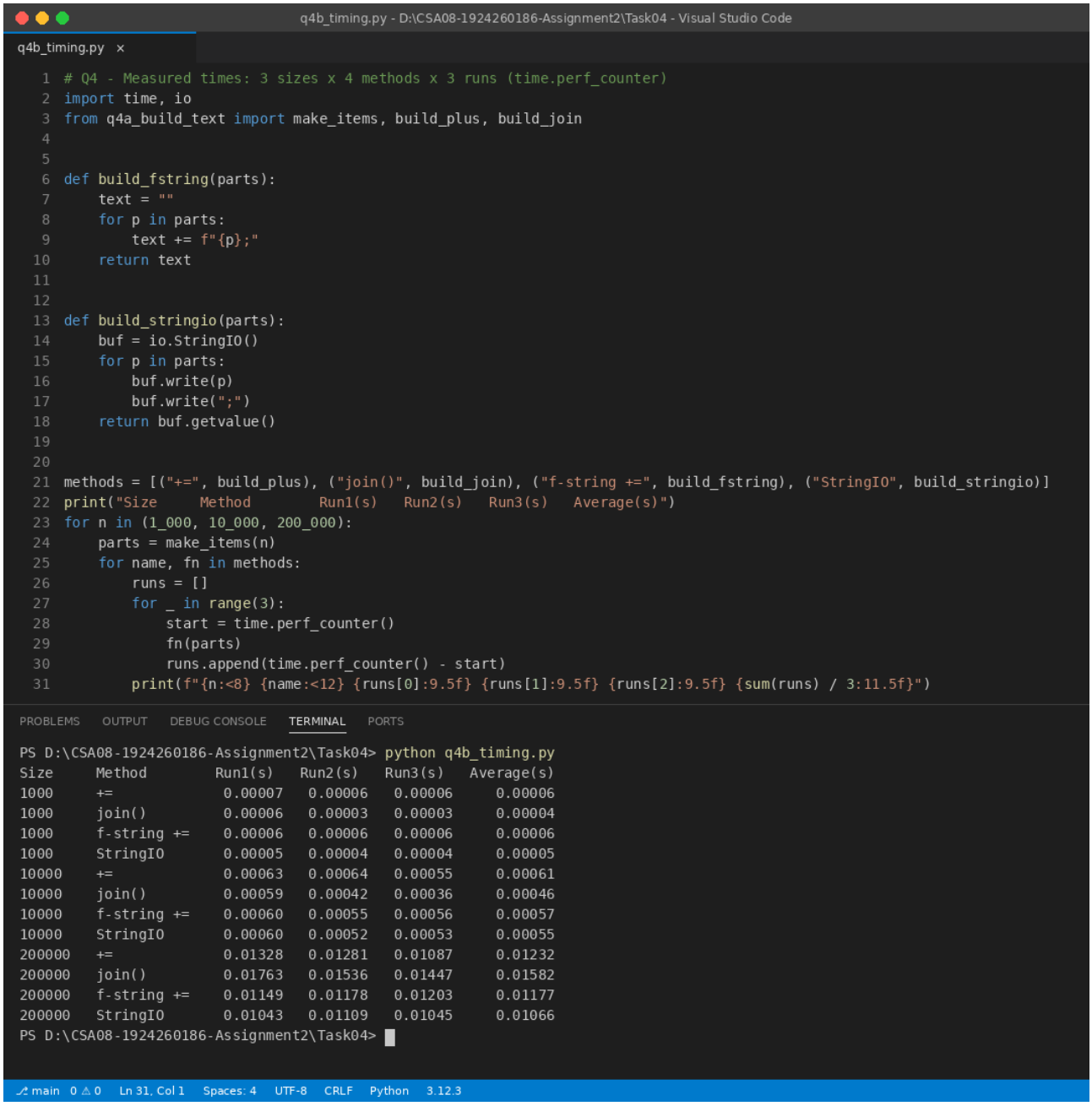
## Q4 - Building long text: + or join() – q4a_build_text.py – the given program: += vs join()

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q4 - Building long text: + or join() – q4b_timing.py – measured times, 3 sizes x 3 runs

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q5 - Functions, parameters and scope – q5a_default_bug.py – mutable default bug and fix

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q5a_default_bug.py - D:\CSA08-1924260186-Assignment2\Task05 - Visual Studio Code

q5a_default_bug.py

```python
# Q5 - Canteen order system: the mutable default argument bug

def add_item_buggy(item, order=[]):          # the list is created ONCE, when def runs
    order.append(item)
    return order


def add_item(item, order=None):              # TODO 1
    if order is None:
        order = []                           # new list for every call
    order.append(item)
    return order


print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"), add_item("Vada"))
print("add_item_buggy.__defaults__ =", add_item_buggy.__defaults__)
print("add_item.__defaults__       =", add_item.__defaults__)

print("--- three customers in sequence (fixed) ---")
for name, items in [("Customer 1", ["Tea", "Vada"]), ("Customer 2", ["Coffee"]),
                    ("Customer 3", ["Idli", "Dosa", "Juice"])]:
    order = None
    for it in items:
        order = add_item(it, order)
    print(f"{name}: {order}")

print("--- same customers with the buggy function ---")
for items in (["Tea", "Vada"], ["Coffee"], ["Idli"]):
    for it in items:
        o = add_item_buggy(it)
    print("  ->", o)
```

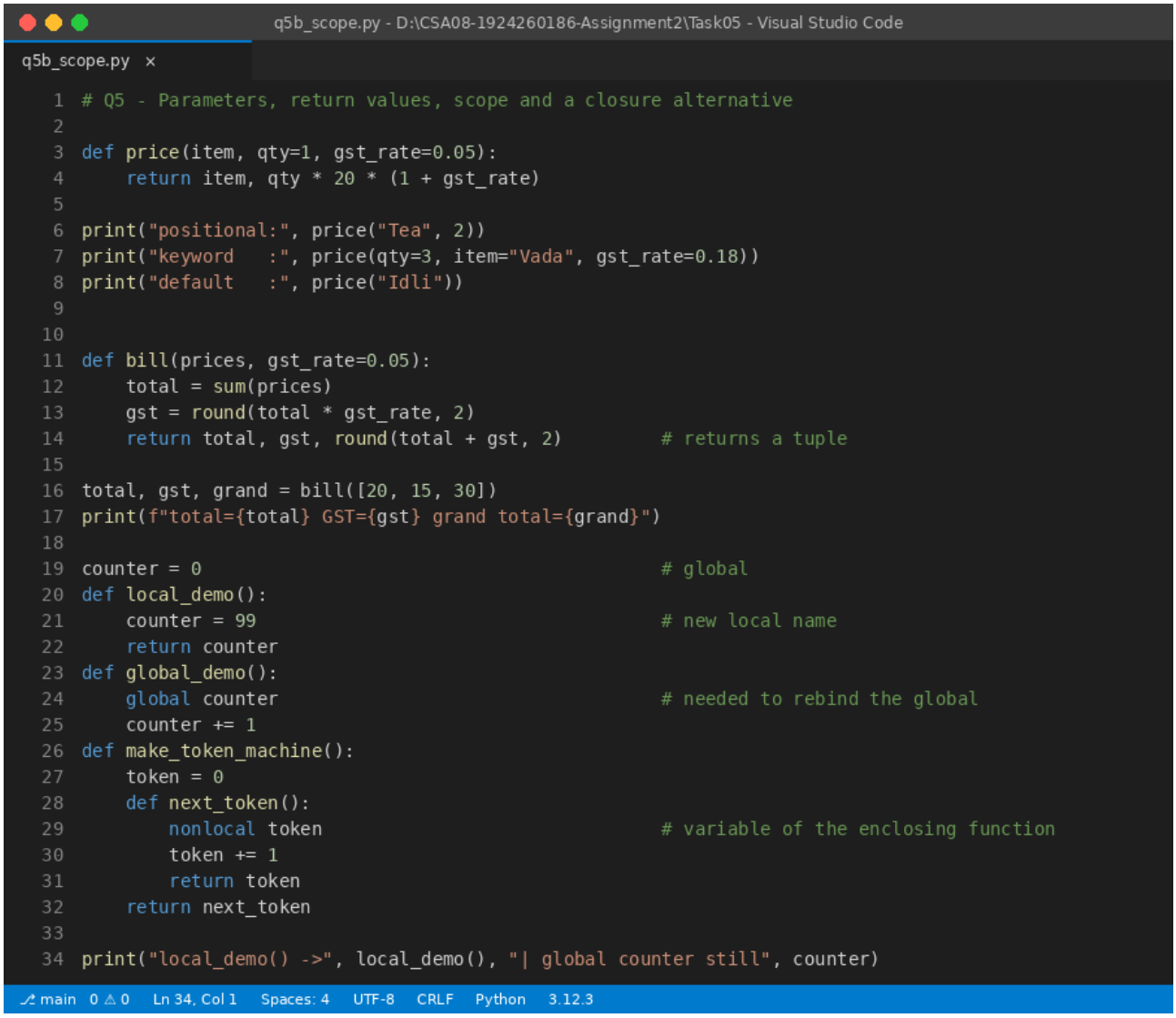
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task05> python q5a_default_bug.py
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea'] ['Vada']
add_item_buggy.__defaults__ = (['Tea', 'Vada'],)
add_item.__defaults__       = (None,)
--- three customers in sequence (fixed) ---
Customer 1: ['Tea', 'Vada']
Customer 2: ['Coffee']
Customer 3: ['Idli', 'Dosa', 'Juice']
--- same customers with the buggy function ---
  -> ['Tea', 'Vada', 'Tea', 'Vada']
  -> ['Tea', 'Vada', 'Tea', 'Vada', 'Coffee']
  -> ['Tea', 'Vada', 'Tea', 'Vada', 'Coffee', 'Idli']
PS D:\CSA08-1924260186-Assignment2\Task05>
```

main 0 ⚠ 0 Ln 32, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

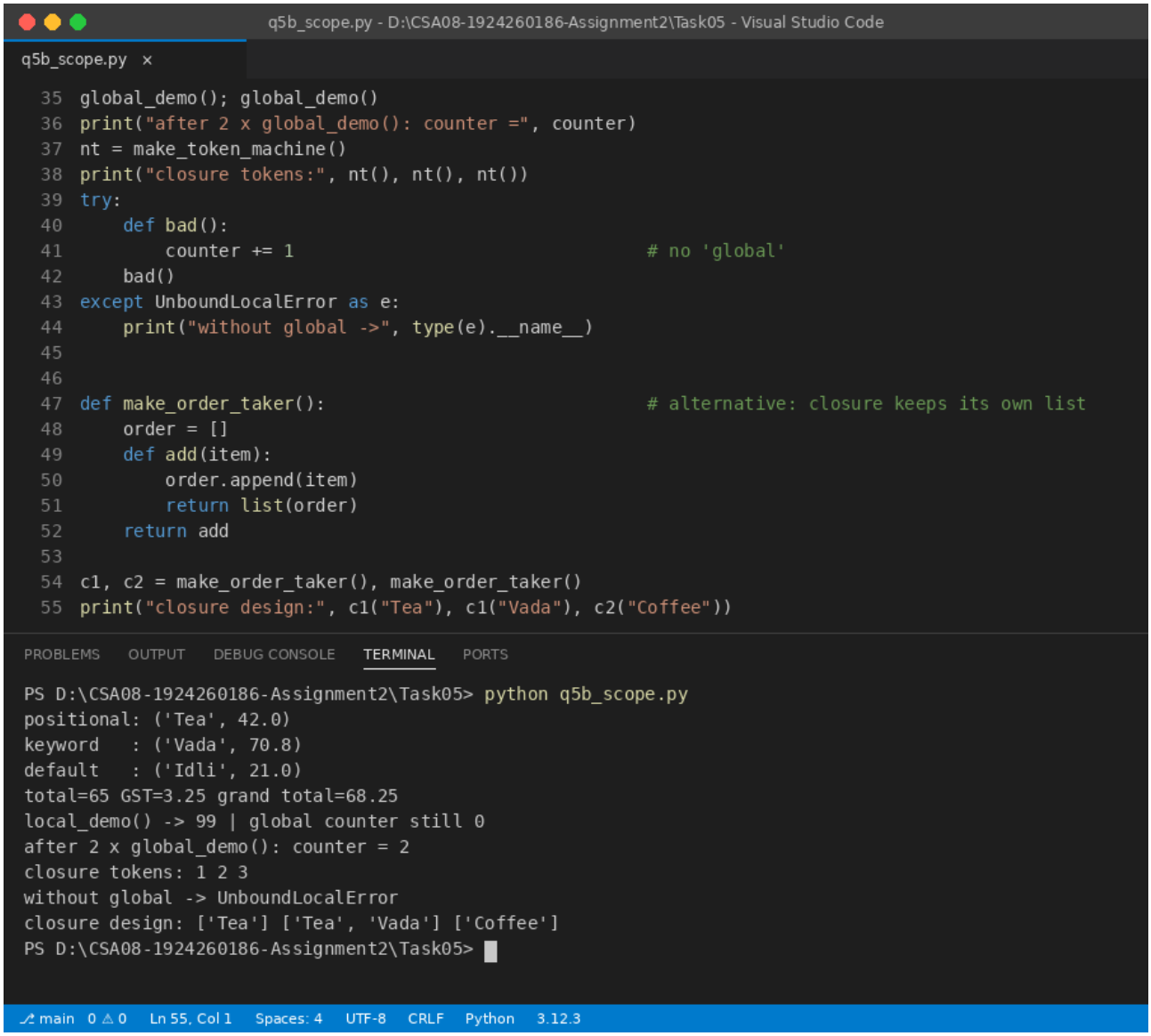
## Q5 - Functions, parameters and scope – q5b_scope.py – parameters, return values, scope, closure (part 1/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q5 - Functions, parameters and scope – q5b_scope.py – parameters, return values, scope, closure (part 2/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q6 - Recursion and its limits – q6a_ways.py – recursive design and memoisation

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q6a_ways.py - D:\CSA08-1924260186-Assignment2\Task06 - Visual Studio Code

q6a_ways.py

```python
# Q6 - Counting ways to climb n temple steps (1 or 2 steps at a time)
from functools import lru_cache

calls = 0


def ways_naive(n):
    """Ways to climb n temple steps taking 1 or 2 steps at a time."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_naive(n - 1) + ways_naive(n - 2)


@lru_cache(maxsize=None)
def ways_memo(n):
    global calls
    calls += 1
    if n <= 1:                                  # TODO 1: same base case
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)   # same recursive case


def ways_loop(n):
    a, b = 1, 1                                 # keeps only the last two values
    for _ in range(n - 1):
        a, b = b, a + b
    return b if n >= 1 else 1


if __name__ == "__main__":
    calls = 0
    print("naive:", ways_naive(25), "calls =", calls)
    calls = 0
    print("memo :", ways_memo(25), "calls =", calls)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task06> python q6a_ways.py
naive: 121393 calls = 242785
memo : 121393 calls = 26
PS D:\CSA08-1924260186-Assignment2\Task06>
```

main 0 ⚠ 0 Ln 36, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

## Q6 - Recursion and its limits – q6b_analysis.py – correctness, call tree, growth

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q6b_analysis.py - D:\CSA08-1924260186-Assignment2\Task06 - Visual Studio Code

q6b_analysis.py

```python
# Q6 - Correctness check, call tree for n = 5 and growth of function calls
import q6a_ways as w

print("n  manual naive memo loop")
for n, manual in {1: 1, 2: 2, 3: 3, 4: 5, 5: 8, 6: 13}.items():
    print(f"{n}  {manual:>6} {w.ways_naive(n):>5} {w.ways_memo(n):>4} {w.ways_loop(n):>4}")


def tree(n, depth=0):
    print("  " * depth + f"ways({n})")
    if n <= 1:
        return 1
    return tree(n - 1, depth + 1) + tree(n - 2, depth + 1)

print("call tree n = 5 ->", tree(5))

print(f"{'n':>3} {'naive calls':>12} {'memo calls':>11} {'2*ways(n)-1':>12}")
for n in (10, 20, 25, 30):
    w.calls = 0
    w.ways_naive(n)
    naive_calls = w.calls
    w.ways_memo.cache_clear()
    w.calls = 0
    w.ways_memo(n)
    print(f"{n:>3} {naive_calls:>12} {w.calls:>11} {2 * w.ways_loop(n) - 1:>12}")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task06> python q6b_analysis.py
n  manual naive memo loop
1       1     1    1    1
2       2     2    2    2
3       3     3    3    3
4       5     5    5    5
5       8     8    8    8
6      13    13   13   13
ways(5)
  ways(4)
    ways(3)
      ways(2)
        ways(1)
        ways(0)
      ways(1)
    ways(2)
      ways(1)
      ways(0)
  ways(3)
    ways(2)
      ways(1)
      ways(0)
    ways(1)
call tree n = 5 -> 8
  n  naive calls  memo calls  2*ways(n)-1
 10          177          11          177
 20        21891          21        21891
 25       242785          26       242785
 30      2692537          31      2692537
PS D:\CSA08-1924260186-Assignment2\Task06>
```

main 0 ⚠ 0 Ln 25, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

## Q6 - Recursion and its limits – q6c_limit.py – recursion limit and deployment decision

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

q6c_limit.py - D:\CSA08-1924260186-Assignment2\Task06 - Visual Studio Code

q6c_limit.py

```python
# Q6 - Deployment decision for n = 1000: recursion limit, loop, timings
import sys, time
import q6a_ways as w

print("recursion limit =", sys.getrecursionlimit())
w.ways_memo.cache_clear()
try:
    w.ways_memo(1000)
    print("memo(1000) worked")
except RecursionError as e:
    print("RecursionError:", e)

sys.setrecursionlimit(5000)                    # raising the limit only moves the problem
w.ways_memo.cache_clear()
print("limit 5000 -> memo(1000) has", len(str(w.ways_memo(1000))), "digits")
print("loop(1000) same value:", w.ways_loop(1000) == w.ways_memo(1000))


def avg(fn, n, reset=None):
    ts = []
    for _ in range(3):
        if reset:
            reset()
        t = time.perf_counter()
        fn(n)
        ts.append(time.perf_counter() - t)
    return sum(ts) / 3

print(f"memoised recursion: {avg(w.ways_memo, 1000, w.ways_memo.cache_clear):.6f} s")
print(f"loop (two values) : {avg(w.ways_loop, 1000):.6f} s")
print("plain recursion at n=1000 needs about 10^209 calls: impossible")
```

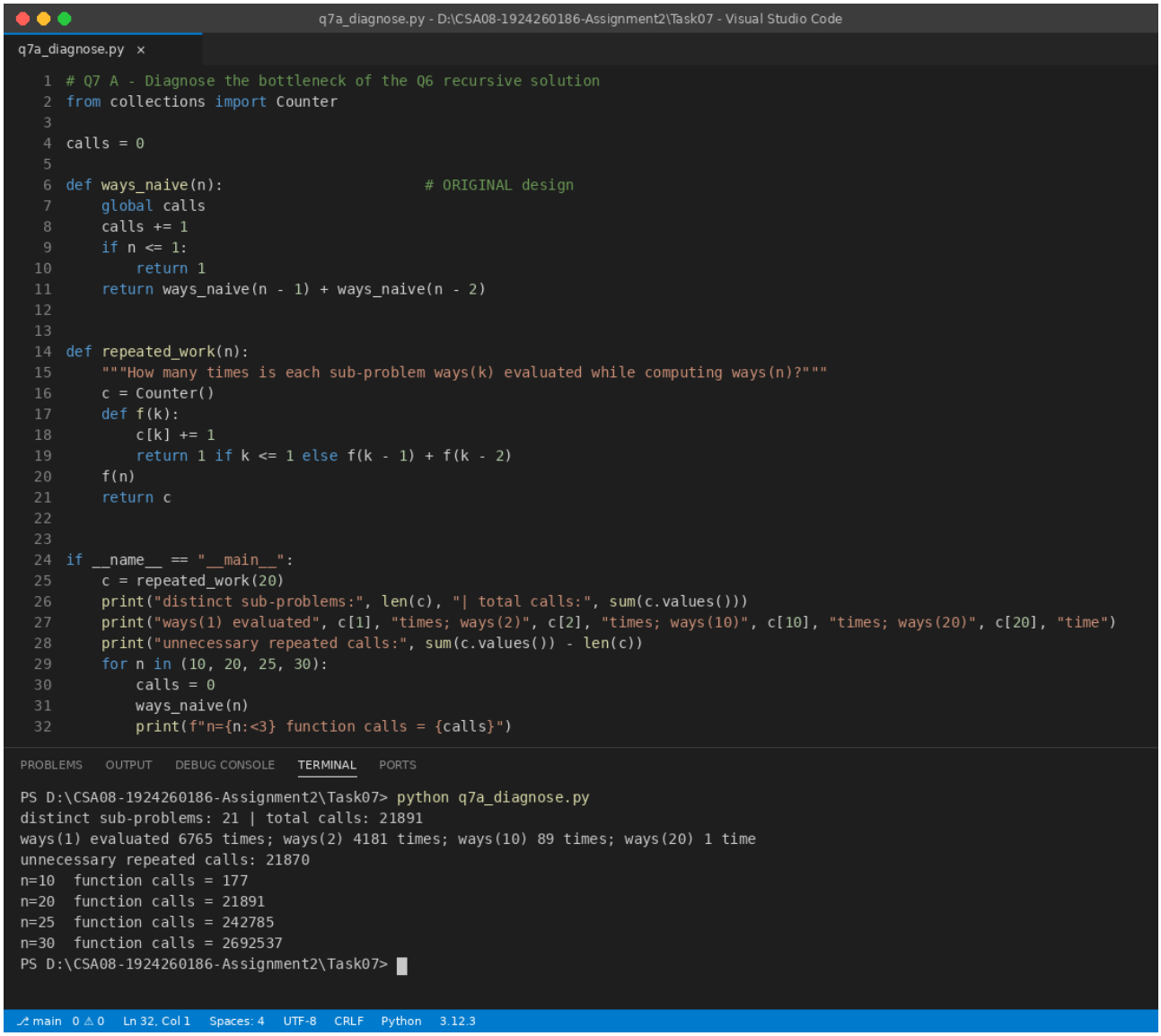
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\CSA08-1924260186-Assignment2\Task06> python q6c_limit.py
recursion limit = 1000
RecursionError: maximum recursion depth exceeded
limit 5000 -> memo(1000) has 209 digits
loop(1000) same value: True
memoised recursion: 0.000255 s
loop (two values) : 0.000035 s
plain recursion at n=1000 needs about 10^209 calls: impossible
PS D:\CSA08-1924260186-Assignment2\Task06>
```

main 0 ⚠ 0 Ln 31, Col 1 Spaces: 4 UTF-8 CRLF Python 3.12.3

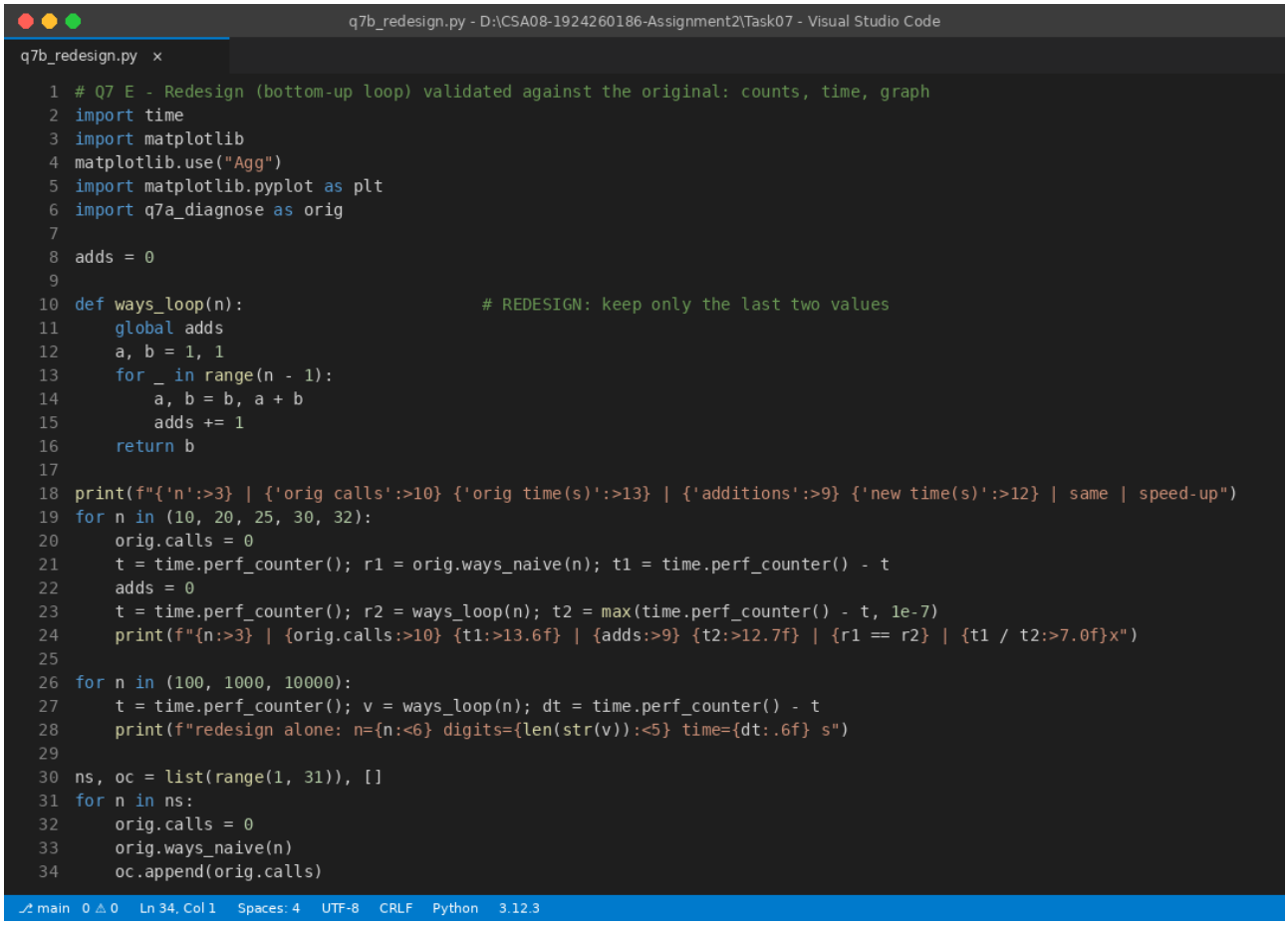
## Q7 - Diagnose, redesign and validate – q7a_diagnose.py – diagnose the bottleneck

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q7 - Diagnose, redesign and validate – q7b_redesign.py – redesign and validate (part 1/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2



## Q7 - Diagnose, redesign and validate – q7b_redesign.py – redesign and validate (part 2/2)

Novia sri .A | 1924260186 | Python 3.12 | SIMATS Assignment 2

